

FORMATION EMBARQUÉE

TD 07 Siemens

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

TD 07 — Siemens TIA Portal : énoncés et corrigés détaillés

Les corrigés LADDER sont donnés en schéma ASCII + description réseau par réseau (à reproduire dans TIA Portal), les corrigés SCL en code complet. Tout se teste avec PLCSIM.

Exercice 1 — Va-et-vient de chariot avec arrêt d'urgence

Énoncé. Un chariot va et vient entre deux fins de course `fdc_gauche` (%I0.2) et `fdc_droite` (%I0.3). Départ par `bp_depart` (%I0.0). Arrêt d'urgence `arret_urgence` (%I0.1, **contact NF câblé** : 1 = tout va bien). Après un arrêt d'urgence, un réarmement par `bp_rearmement` (%I0.4) est obligatoire avant tout redémarrage. Sorties : `mvt_droite` (%Q0.0), `mvt_gauche` (%Q0.1).

Corrigé

Table des variables :

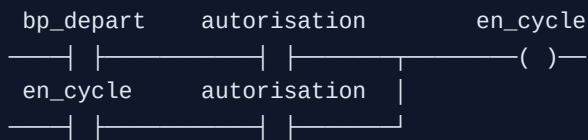
Nom	Type	Adresse	Commentaire
<code>bp_depart</code>	Bool	%I0.0	NO
<code>arret_urgence</code>	Bool	%I0.1	NF câblé : 1 = OK, 0 = AU appuyé
<code>fdc_gauche</code> / <code>fdc_droite</code>	Bool	%I0.2 / %I0.3	fins de course
<code>bp_rearmement</code>	Bool	%I0.4	NO
<code>autorisation</code>	Bool	%M0.0	mémoire de réarmement
<code>en_cycle</code>	Bool	%M0.1	cycle demandé
<code>sens_droite</code>	Bool	%M0.2	mémoire de sens
<code>mvt_droite</code> / <code>mvt_gauche</code>	Bool	%Q0.0 / %Q0.1	contacteurs

Réseau 1 — Autorisation (réarmement obligatoire) :

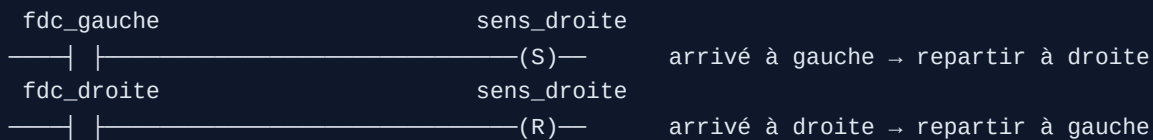
```
bp_rearmement  arret_urgence          autorisation
—| |—————| |—————|—————( )—
autorisation  arret_urgence          |
—| |—————| |—————|—————|
```

`autorisation` s'établit par le réarmement (si l'AU est relâché) et se maintient **tant que** l'AU reste relâché : le moindre AU la fait retomber, et elle ne revient pas seule — il faut réappuyer sur réarmement.

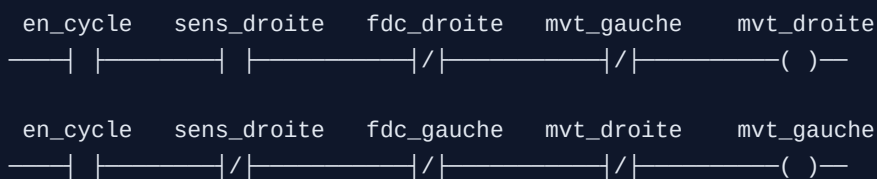
Réseau 2 — Demande de cycle (auto-maintien) :



Réseau 3 — Mémoire de sens (basculer S/R par les fins de course) :



Réseaux 4 et 5 — Sorties avec verrouillage mutuel :



Points de correction (les non-négociables) : 1. **AU en contact NF** : fil coupé = AU déclenché (sécurité positive). Un AU en NO qui perd son fil ne protégera plus jamais. 2. **Réarmement obligatoire** : l'AU relâché ne suffit pas à repartir (norme machine : pas de redémarrage automatique après coupure sécurité). 3. **Verrouillage mutuel** des deux sorties (**mvt_droite** interdit **mvt_gauche** et réciproquement) : deux contacteurs de sens fermés ensemble = court-circuit. 4. Chaque sortie est aussi coupée par **sa** fin de course.

Exercice 2 — FB « Moyenne_Glissante » en SCL

Énoncé. FB avec Array de 10 Real, entrée **nouvelle_mesure**, entrée **echantillonner** (à traiter sur front), sortie **moyenne**. L'instancier deux fois.

Corrigé

Interface du FB :

Section	Nom	Type
Input	nouvelle_mesure	Real
Input	echantillonner	Bool
Output	moyenne	Real
Static	mesures	Array[0..9] of Real
Static	index	Int
Static	nb_valides	Int
Static	echant_prec	Bool
Temp	i	Int
Temp	somme	Real

Corps en SCL :

```
// Détection de front montant sur la commande d'échantillonnage :
// un FB est appelé À CHAQUE CYCLE ; sans front, on stockerait la même
// mesure des centaines de fois par seconde.
IF #echantillonner AND NOT #echant_prec THEN
    #mesures[#index] := #nouvelle_mesure;
    #index := (#index + 1) MOD 10;           // tampon circulaire (cf. TD 01 !)
    IF #nb_valides < 10 THEN
        #nb_valides := #nb_valides + 1;     // démarrage : moyenne sur n < 10
    END_IF;
END_IF;
#echant_prec := #echantillonner;

// Moyenne sur les échantillons réellement présents
#somme := 0.0;
FOR #i := 0 TO #nb_valides - 1 DO
    #somme := #somme + #mesures[#i];
END_FOR;

IF #nb_valides > 0 THEN
    #moyenne := #somme / INT_TO_REAL(#nb_valides);
ELSE
    #moyenne := 0.0;                       // jamais de division par zéro
END_IF;
```

Double instanciation (dans OB1) : appeler le FB deux fois en créant deux **DB d'instance** distincts (**DB_Moyenne_Temp**, **DB_Moyenne_Pression**) — chaque instance garde SON tableau et SON index. C'est exactement « une classe, deux objets » (TD 02).

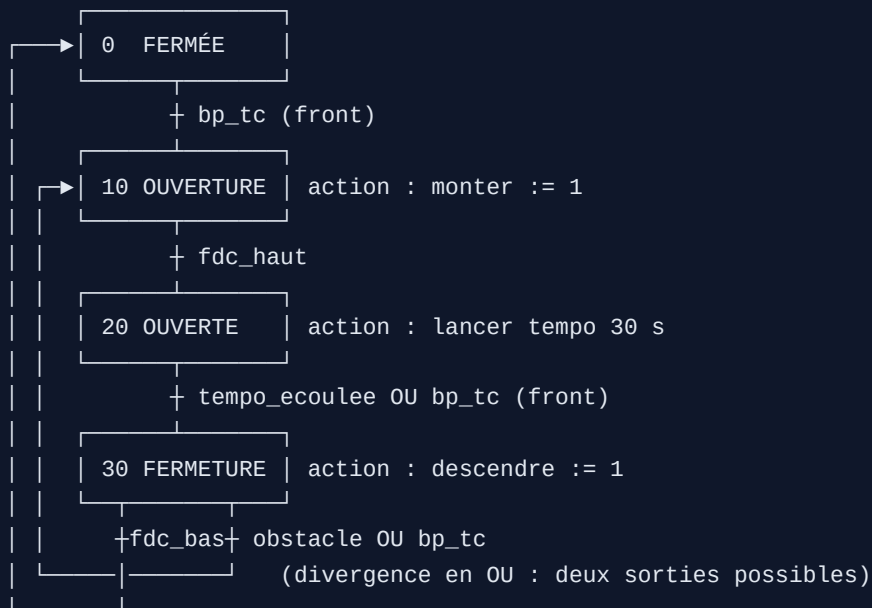
Points de correction : la détection de front interne (l'erreur n°1 des débutants en FB) ; le démarrage progressif (**nb_valides**) ; la garde de division ; la conversion explicite **INT_TO_REAL** (SCL ne convertit pas silencieusement).

Exercice 3 — GRAFCET porte de garage → SCL

Énoncé. Une impulsion télécommande (`bp_tc`) ouvre la porte ; en haut, temporisation de 30 s puis fermeture automatique ; pendant la fermeture, la cellule `obstacle` fait rouvrir ; fins de course `fdc_haut` , `fdc_bas` .

Corrigé

GRAFCET (à dessiner d'abord — c'est lui qui est noté) :



Implémentation SCL (FB, avec un `TON` nommé `tempo` en Static) :

```

CASE #etape OF
  0: // FERMÉE
    #monter := FALSE; #descendre := FALSE;
    IF #front_tc THEN #etape := 10; END_IF;

  10: // OUVERTURE
    #monter := TRUE; #descendre := FALSE;
    IF #fdc_haut THEN #etape := 20; END_IF;

  20: // OUVERTE (tempo de fermeture auto)
    #monter := FALSE; #descendre := FALSE;
    #tempo(IN := TRUE, PT := T#30s);
    IF #tempo.Q OR #front_tc THEN
      #tempo(IN := FALSE);           // réarmer la tempo en la quittant
      #etape := 30;
    END_IF;

  30: // FERMETURE
    #descendre := TRUE; #monter := FALSE;
    IF #obstacle OR #front_tc THEN
      #etape := 10;                 // priorité : on ROUVRE
    ELSIF #fdc_bas THEN
      #etape := 0;
    END_IF;
END_CASE;

// hors CASE : sécurité absolue, quel que soit l'état
IF #monter AND #descendre THEN    // ne doit JAMAIS arriver
  #monter := FALSE; #descendre := FALSE; #etape := 0;
END_IF;

```

Points de correction : une étape = un cas, les actions écrites dans **chaque** étape (pas d'action fantôme qui « reste collée ») ; l'ordre des conditions en étape 30 — l'obstacle est testé **avant** `fdc_bas` (priorité à la sécurité) ; la tempo réarmée en quittant l'étape 20 ; le verrouillage final monter/descendre.

Exercice 4 — Écran HMI

Énoncé. Écran avec bouton MARCHE, voyant moteur, champ consigne vitesse, alarme « défaut thermique ».

Corrigé (démarche attendue, à réaliser dans WinCC)

- Variables HMI** liées aux variables API : `cmd_marche` (Bool), `etat_moteur` (Bool), `consigne_vitesse` (Int, limites 0..1500), `defaut_thermique` (Bool).
- Bouton MARCHE** : événements « Appuyer » → mise à 1 de `cmd_marche`, « Relâcher » → mise à 0 (bouton à impulsion). **Piège de conception** : ne jamais faire un bouton HMI qui « reste à 1 » — si la liaison tombe, l'ordre reste bloqué. La logique de maintien appartient à l'automate (auto-maintien de l'exercice 1), pas à l'écran.

3. **Voyant** : un cercle dont l'apparence est animée par `etat_moteur` — et ce doit être le **retour réel** (contact du contacteur), pas la recopie de la commande : un voyant qui affiche « ce qu'on a demandé » au lieu de « ce qui se passe » masque les pannes.
 4. **Champ d'E/S** consigne : format numérique, **limites min/max déclarées** (l'automate revalide de son côté — défense en profondeur).
 5. **Alarme TOR** sur `defaut_thermique` : classe « Errors », texte explicite (« Défaut thermique moteur M1 — vérifier relais F2 »), acquittement requis ; fenêtre des alarmes sur l'écran.
 6. Test : simulateur HMI + PLCSIM ensemble, scénario complet (marche, changement de consigne, déclenchement du défaut, acquit).
-

Auto-évaluation avant le TP 3

Sans notes : justifier NF pour un AU ; dessiner l'auto-maintien ; expliquer front vs niveau dans un FB appelé chaque cycle ; différence FC/FB/DB ; pourquoi la logique de sécurité vit dans l'automate et jamais dans l'HMI.

➡ Passe au **TP 3 — Station de remplissage**.